SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

Detection of Cyber Attack in Network using Machine Learning Techniques

INT 500

PROJECT REPORT

Submitted by

DAGUMATI SRINATH- E0220017

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Cyber Security & Internet of Things)

Sri Ramachandra Faculty of Engineering and Technology

Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai -600116

APRIL, 2024

Detection of Cyber Attack in Network using Machine Learning Techniques

INT 500

PROJECT REPORT

Submitted by

DAGUMATI SRINATH – E0220017

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Cyber Security & Internet of Things)

Sri Ramachandra Faculty of Engineering and Technology

Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai -600116

APRIL, 2024

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Certified that this project report **"Detection of Cyber Attack in Network using Machine Learning Techniques"** is the bonafide record of work done by **"DAGUMATI SRINATH – E0220017"** who carried out the internship work under my supervision.

Signature of the Supervisor

Signature of Programme Coordinator

Ms. Yamini K

Lecturer,

Department of Cybersecurity and IoT

Sri Ramachandra Faculty of Engineering and Technology,

SRIHER, Porur, Chennai-600 116.

Dr. Jayanthi G

Associate Professor,

Department of Cybersecurity and IoT

Sri Ramachandra Faculty of Engineering and Technology,

SRIHER, Porur, Chennai-600 116.

Evaluation Date:

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

ACKNOWLEDGEMENT

I express my sincere gratitude to our Programme Coordinator **Dr. Jayanthi G** for their support and for providing the required facilities for carrying out this study.

I wish to thank my faculty supervisor(s), **Ms. Yamini K,** Department of Artificial Intelligence and Machine Learning, Sri Ramachandra faculty of Engineering and Technology and **INDUSTRY MENTOR, DESIGNATION -IF APPLICABLE** for extending help and encouragement throughout the project. Without his/her continuous guidance and persistent help, this project would not have been a success for me.

I am grateful to all the members of Sri Ramachandra Faculty of Engineering and Technology, my beloved parents and friends for extending the support, who helped us to overcome obstacles in the study.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**8**
	LIST OF TABLES	
	LIST OF FIGURES	
	LIST OF SYMBOLS	
1	**INTRODUCTION**	**9**
	1.1 Brief About Project	**9**
	1.2 Data preprocessing	**10**
	1.2.1 Deletion	**10**
	1.2.2 Mean/Median Imputation	**10**
	1.2.3 Label Encoding	**11**
	1.2.4 Outlier	**11**
	1.3 Machine learning-	**13**
	1.3.1 Supervised Machine Learning	**14**
	1.3.2 Unsupervised Machine Learning	**15**
	1.3.3 Semi-Supervised Learning	**16**
	1.3.4 Logistic regression	**17**
	1.3.5 K-Nearest Neighbor	**18**
	1.3.6 Decision Tree	**19**
2	**LITERATURE REVIEW**	**21**
3	**PROPOSED METHODOLOGY**	**25**
4	**TOOLS AND TECHNIQUES**	**26**

5 IMPLEMENTATION 27

4.1 Importing libraries **27**

4.2 Analyse a dataset **27**

4.3 Unique attacks **28**

4.4 Summary of a dataframe **28**

4.5 Data Formatting **29**

4.6 Data Formatting Train Dataset **29**

4.7 Outliers **30**

4.8 Removing outlier **30**

4.9 Train Test split **31**

6 RESULTS AND DISCUSSIONS 32

5.1 Logistic Regression **32**

5.2 Confusion Matrix **32**

5.3 k-nearest neighbors **33**

5.4 Confusion Matrix **33**

5.5 DecisionTree **34**

5.6 Confusion Matrix **34**

5.7 Streamlit **35**

7 REFERENCES 36

6.1 Journal Reference **36**

6.2 Web Reference **37**

8 WORKLOG 38

LIST OF FIGURES

FIG NO	FIGURE NAME	PAGE NO
4.1	Importing libraries	27
4.2	Reading dataset	27
4.3	Dataset unique attacks	28
4.4	Dataset info	28
4.5	Encoding Categorical values of train dataset	29
4.6	Encoding Categorical values of test dataset	29
4.7	Outliers	30
4.8	Removing outlier	30
4.9	Train Test split	31
5.1	Logistic Regression	32
5.2	Confusion Matrix	32
5.3	k-nearest neighbors	33
5.4	Confusion Matrix	33
5.5	DecisionTree	34
5.6	Confusion Matrix	34
5.7	Streamlit	35

ABSTRACT

Technological advancements have brought significant benefits, but they also introduce new security challenges. Protecting sensitive data, securing information storage, and ensuring data availability are crucial concerns. Cyberterrorism has become a major threat, jeopardizing both individual and national security. Malicious actors, including criminal organizations, skilled hackers, and cyber activists, pose a significant risk. To combat cyber threats, Intrusion Detection Systems (IDS) have been created to identify and prevent cyberattacks. This research employed a specific machine learning algorithm to detect a type of cyberattack within a dataset. While achieving promising accuracy, the ability to precisely identify real threats needs improvement. The paper proposes exploring alternative algorithms that might offer similar or even better performance.

CHAPTER 1

INTRODUCTION

1.1 Brief About Project

The rapid rise of interconnected devices and data exchange through technologies like the Internet of Things (IoT), cloud computing, and 5G networks has created a complex security challenge. This interconnectedness, while beneficial, exposes vast amounts of sensitive data on untrusted networks. Traditional security methods struggle to keep pace with the evolving tactics of attackers. Machine Learning (ML) offers a promising solution for intrusion detection. ML algorithms can analyze vast amounts of network traffic data and learn to identify anomalies or patterns indicative of malicious activity. This adaptability allows them to detect novel attacks that traditional signature-based systems might miss. ML offers several benefits for intrusion detection: improved accuracy, adaptability to new attack patterns, and real-time detection capabilities. However, challenges remain. The effectiveness of ML models depends on high-quality training data, and they can generate false positives. Additionally, training and running these models can be computationally expensive. Looking ahead, as ML techniques advance and computational resources become more accessible, their role in network security will become even more critical. Research is exploring areas like deep learning for more sophisticated anomaly detection and AI for automated incident response. By harnessing the power of Machine Learning, we can build robust and adaptable defenses against the ever-evolving threat landscape of cyberattacks..

1.2 Data preprocessing

Before diving into analysis, raw data often requires some cleaning and preparation. This process, known as data preprocessing, plays a critical role in data mining and machine learning projects. Data collection methods can introduce inconsistencies like out-of-range

values, missing entries, and illogical combinations. Preprocessing techniques address these issues, ensuring the data is high quality and suitable for analysis. The chosen preprocessing steps can significantly impact the conclusions drawn from the analysis. Data quality and its representation are paramount for reliable results. In machine learning, particularly within the field of computational biology, data preprocessing is often considered the most crucial stage of a project. When data contains a high proportion of irrelevant, redundant, or noisy information, it becomes more challenging for machine learning models to learn effectively during the training phase. Data preparation and filtering can be time-consuming due to the sheer volume of data involved. However, the investment in preprocessing is essential for robust and reliable results. Common methods used in data preprocessing include cleaning (removing errors), instance selection (choosing relevant data points), normalization (scaling data), one-hot encoding (representing categorical data numerically), data transformation (changing data format), feature extraction (creating new features), and feature selection (choosing the most informative features). By applying these techniques, we can ensure our data is clean, consistent, and ready to be used for effective analysis and machine learning model training.

Data preprocessing techniques:

1.2.1 Deletion:

This is a straightforward approach where data points with missing values are simply removed from the dataset. However, deletion can be problematic if the amount of missing data is significant, as it can lead to a loss of valuable information and potentially affect the overall representativeness of the data**.**

1.2.2 Mean/Median Imputation:

Replaces missing values with the average (mean) or middle value (median) of the feature (column) they belong to. This is a simple and fast approach but may not be suitable for skewed data distributions.

1.2.3 Label Encoding : `

Machine learning models excel at processing numerical data, but what about information like clothing sizes (S, M, L) or income levels (low, medium, high)? These are categorical variables, and to make them usable for our models, we need a data preprocessing technique called label encoding. Here's how it works: Imagine a dataset with a column for "shirt size" containing values like "small," "medium," and "large." Label encoding takes each unique category ("small," "medium," "large") and assigns it a unique numerical label. It might assign "small" a value of 1, "medium" a value of 2, and "large" a value of 3. This transforms the text labels into a numerical format that machine learning models can understand and work with effectively. One of the benefits of label encoding is that it can preserve the inherent order of the categories, if it exists. Continuing with the shirt size example, the assigned values (1, 2, 3) reflect the increasing size of the shirt.

1.2.4 Outlier:

In the world of data analysis, outliers are data points that stand out from the rest, like the lone bright green apple in a basket of red ones. These "outliers" are important to identify because they can significantly impact the results of statistical analyses. The process of finding these outliers is called outlier mining. Outliers can skew the average (mean) and other measures of what's typical in the data, leading to misinterpretations. They can also influence the outcome of tests used to determine statistical significance. So, what causes these outliers to appear? There are several culprits: Measurement Errors: Faulty instruments or mistakes during data collection can introduce outliers. Sampling Errors: If the sampling method isn't perfect, outliers might sneak into the data set. Natural Variation: Sometimes, the very nature of what's being studied can lead to outliers. For example, some systems naturally have extreme values. Human Error: Typos or mistakes

during data entry can create outliers. Unexpected Events: Experiments can be disrupted by unforeseen events or equipment malfunctions, leading to outliers in the data. Mixing Populations: Combining data from different groups (populations) with distinct characteristics can create outliers. Intentional Outliers: In rare cases, outliers might be added on purpose to test how well statistical methods handle them. By understanding these potential causes, data analysts can be more critical when analyzing data. Techniques for outlier detection help identify these odd ones out, allowing for informed decisions about how to handle them and ensure the integrity of the overall analysis. profile picture without points Outliers: Disrupting the Data Landscape In data analysis, outliers are data points that deviate significantly from the rest, disrupting the overall picture. Identifying these outliers is crucial because they can heavily influence the results of statistical analyses. The process of finding them is called outlier mining. Outliers can distort the average (mean) and other measures of central tendency, leading to misleading interpretations of the data. Additionally, they can influence the outcome of tests designed to assess statistical significance. The presence of outliers can stem from various sources: Measurement or Sampling Errors: Faulty instruments, mistakes during data collection, or issues with the sampling method can introduce outliers. Natural Variability: Inherent differences within the data itself can naturally produce outliers. Some systems inherently exhibit extreme values. Data Entry Errors: Human mistakes during data entry can introduce outliers.

1.3 Machine learning :

Machine learning (ML) is a rapidly evolving field within artificial intelligence (AI) that empowers computers to learn and improve without explicit programming. Imagine teaching a computer to recognize patterns in data, just like a child learns to identify a dog from a cat. This is the essence of machine learning. Here's how it works: Data is King: Machine learning algorithms rely heavily on data. The more data you have, the better the model can learn. This data can be anything from text and images to sensor readings and

financial transactions. Training the Model: Data scientists prepare the data and feed it into the machine learning algorithm. The algorithm analyzes the data, identifying patterns and relationships between different data points. This process is called training. Making Predictions: Once trained, the algorithm can use its newfound knowledge to make predictions on new, unseen data. For example, a machine learning model trained on images of handwritten digits can predict the digit in a new handwritten image it has never seen before. The Benefits of Machine Learning: Machine learning offers a wide range of benefits across various industries: Automation: Automating repetitive tasks, freeing up human resources for more complex work. Improved Decision Making: Extracting insights from data that humans might miss, leading to better decision-making. Personalization: Recommending products or services based on individual preferences, enhancing customer experience. Fraud Detection: Identifying suspicious activity and preventing fraudulent transactions. Scientific Discovery: Accelerating scientific research by analyzing vast amounts of data to uncover hidden patterns. The Future of Machine Learning: As machine learning continues to evolve, its applications will become even more pervasive. We can expect to see advancements in areas like: Deep Learning: A powerful subfield of machine learning inspired by the structure and function of the human brain.

Machine learning methods :

1.3.1 Supervised Machine Learning :

Supervised machine learning is a powerful technique within artificial intelligence (AI) where algorithms learn from labeled examples. Imagine a student learning with a teacher who provides labeled data (correct answers) to guide their understanding. In supervised learning, the data acts as the teacher, and the algorithm is the student.

Here's how it works:

Preparation is Key: The data preparation stage is crucial. We provide the algorithm with a dataset containing labeled input data points. Each data point has a corresponding desired

output, like a category label (spam/not spam) or a numerical value (house price prediction).

Learning from Examples: The algorithm analyzes the labeled data, identifying patterns and relationships between the input features and the desired outputs. This process is called training. As the algorithm processes more data, it refines its internal model, adjusting its parameters to improve its accuracy in predicting the desired output.

Testing and Refinement: Once trained, the algorithm is evaluated on a separate set of unseen data. This helps assess how well the model generalizes its learned knowledge to new situations. Techniques like cross-validation help prevent the model from overfitting (memorizing the training data) or underfitting (failing to capture the underlying patterns).

Real-World Applications: Supervised machine learning finds applications in a vast array of fields:

Spam Filtering: Classifying emails as spam or legitimate.

Image Recognition: Identifying objects or scenes within an image.

Fraud Detection: Spotting suspicious financial transactions.

Stock Price Prediction: Forecasting future stock market trends (with inherent limitations).

Common Supervised Learning Techniques:

Linear Regression: Predicts a continuous output value based on a linear relationship with the input features.

Logistic Regression: Classifies data points into discrete categories (e.g., spam/not spam).

Support Vector Machines (SVMs): Creates a hyperplane to separate data points belonging to different categories.

Random Forests: Combines multiple decision trees to improve prediction accuracy and reduce overfitting.

Naive Bayes: Classifies data based on the assumption of independence between features (a simplifying assumption that may not always hold true).

Neural Networks: Inspired by the structure and function of the human brain, these complex algorithms can learn intricate patterns from large datasets.

1.3.2 **Unsupervised Machine Learning**: Unveiling Hidden Patterns

In the realm of machine learning, unsupervised learning takes a different approach compared to its supervised counterpart. Here, algorithms delve into the unknown, exploring unlabeled data. Imagine being presented with a box of uncategorized objects - that's the essence of unsupervised learning. The goal? To uncover hidden patterns and groupings within the data, all without any predefined labels or instructions.

This ability to discover inherent structures and similarities makes unsupervised learning a valuable tool for various tasks:

Exploratory Data Analysis: Unsupervised learning techniques can help us understand the underlying characteristics of a dataset, identifying potential relationships and trends.

Customer Segmentation: By grouping customers based on their shared features and behaviors, unsupervised learning can inform targeted marketing strategies.

Image and Pattern Recognition: Unsupervised algorithms can identify objects or patterns within images, even without prior training on labeled examples.

Dimensionality Reduction: When dealing with high-dimensional data (many features), unsupervised learning techniques like Principal Component Analysis (PCA) can help reduce the number of features while preserving the most important information.

Common Unsupervised Learning Techniques:

K-Means Clustering: This popular technique groups data points into a predefined number of clusters based on their similarity.

Probabilistic Clustering Methods: These methods assign probabilities to each data point belonging to different potential clusters, offering a more flexible approach compared to k-means clustering.

Neural Networks: While often used in supervised learning, neural networks can also be employed in unsupervised settings to discover hidden patterns within complex data.

1.3.3 Semi-Supervised Learning : Bridging the Gap

In the world of machine learning, data labeling can be a time-consuming and expensive endeavor. Supervised learning thrives on labeled data, but what if obtaining enough labeled data is a challenge? This is where semi-supervised learning steps in, offering a bridge between supervised and unsupervised approaches.

Semi-supervised learning is a powerful technique that leverages both labeled and unlabeled data during the training process. Imagine having a small set of carefully labeled examples, along with a much larger collection of unlabeled data. Semi-supervised learning algorithms can utilize the knowledge from the labeled data to extract meaningful features and patterns, and then apply those insights to the vast unlabeled dataset.

This approach offers several advantages:

Overcoming Data Scarcity: When labeled data is limited, semi-supervised learning can significantly improve model performance by incorporating the additional information from the unlabeled data.

Reduced Labeling Costs: The need for a massive amount of labeled data can be a significant bottleneck in supervised learning. By effectively using unlabeled data, semi-supervised learning reduces the amount of human effort required for data labeling.

Here's a breakdown of how it works:

A Little Goes a Long Way: The training process begins with a relatively small set of labeled data points. These labeled examples provide the foundation for the algorithm's learning process.

Unlocking the Potential of Unlabeled Data: The vast amount of unlabeled data comes into play next. The algorithm analyzes this data, searching for hidden patterns and relationships that can be used to improve its understanding of the overall data landscape.

Synergy for Enhanced Learning: By combining the insights from both labeled and unlabeled data, the semi-supervised learning algorithm refines its internal model, ultimately leading to more accurate predictions.

1.3.4 Logistic regression :

Logistic regression is a powerful tool in machine learning, specifically designed for binary classification tasks. Here, the goal is to predict the outcome of a situation that can only fall into two categories, like "spam" or "not spam" for an email, or "passed" or "failed" for an exam.

Unlike linear regression, which predicts continuous values, logistic regression focuses on categorical dependent variables. These variables can only take on specific values, often represented as 0 or 1 (but can also be "yes" or "no"). However, instead of simply outputting 0 or 1, logistic regression goes a step further.

The Power of Probability:

Logistic regression utilizes a mathematical function called the sigmoid function (also known as the logistic function). This S-shaped function takes any real number as input and transforms it into a probability value between 0 and 1. Imagine feeding the model data about an email. The output wouldn't be a simple "spam" or "not spam," but a probability value indicating how likely it is to be spam.

The Threshold Effect:

To make a final classification decision, a threshold value is often used. This threshold is typically set at 0.5, but it can be adjusted based on the specific problem. Any data point with a predicted probability above the threshold is classified into one category (e.g., spam), while those below the threshold fall into the other (not spam).

Beyond Binary: Exploring Other Types:

While logistic regression is often used for binary classification, there are variations that can handle more complex scenarios:

Binomial: The classic two-category case, like spam detection.

Multinomial: Handles situations with three or more unordered categories, such as classifying emails as "spam," "work," or "personal."

Ordinal: Used when the categories have a natural order, like predicting customer satisfaction ratings (low, medium, high).

Logistic regression is a versatile tool for various c

1.3.5 K-Nearest Neighbor(KNN) Algorithm :

KNN falls under the umbrella of supervised learning, meaning it learns from labeled data. This data consists of examples where each data point has a corresponding label or category (e.g., spam/not spam for emails). KNN excels in areas like pattern recognition, data mining, and intrusion detection.

One of its key strengths is being non-parametric. Unlike some algorithms that make assumptions about the data's underlying structure, KNN makes no such assumptions. This makes it adaptable to various data types, including those that might not follow a neat mathematical distribution.

The Power of Proximity: How KNN Makes Predictions

Imagine you have a dataset with labeled data points. These data points can be visualized as existing in a space with multiple dimensions (depending on the number of features). KNN works by looking at the k nearest neighbors of a new, unlabeled data point.

Here's the core concept: similar data points tend to be close together in this multidimensional space. By analyzing the labels of the k nearest neighbors, KNN can make an educated guess about the label of the new data point.

Visualizing KNN in Action:

Imagine a dataset with two features, where data points are colored red or green. A new, unlabeled data point is introduced. By looking at its closest neighbors (k neighbors) and their labels, KNN can predict the class (red or green) for this new data point.

Why Use KNN?

KNN offers several advantages:

Simplicity: KNN is easy to understand and implement, making it a great starting point for beginners in machine learning.

Flexibility: It can handle both numerical and categorical data, offering a wider range of applicability.

Non-parametric: No assumptions about the data's distribution are required.

Outlier Resilience: KNN is less sensitive to outliers compared to some other algorithms.

1.3.6 Decision Tree :

In the world of machine learning, decision trees stand out for their versatility and interpretability. Imagine a flowchart that helps you make decisions based on a series of questions - that's the essence of a decision tree algorithm. It's a powerful tool that can be

used for both classification (predicting categories) and regression (predicting continuous values).

Understanding the Structure:

Decision trees have a hierarchical structure, resembling an actual tree. Here's a breakdown of the key components:

Root Node: The starting point, representing the initial question or feature considered.

Internal Nodes (Decision Nodes): These nodes ask questions about specific features in the data. Each branch emerging from these nodes represents a possible answer to the question.

Leaf Nodes (Terminal Nodes): These nodes represent the final predictions or decisions. They don't have any further branches.

Branches (Edges): These connect nodes, illustrating the flow of decision-making based on the answers to questions.

Building the Tree:

The construction of a decision tree involves a process called splitting. At each internal node, the algorithm chooses a feature and a specific value (threshold) to split the data into subsets. This splitting continues until a stopping criteria is met, like reaching a certain level of purity (meaning all data points in a leaf node belong to the same class).

Benefits of Decision Trees:

Interpretability: Unlike some complex machine learning models, decision trees are easy to understand and visualize. You can trace the path through the tree to see how a particular prediction is made.

CHAPTER 2

Literature Review

Title ; Detection of Cyber Attack in Network using Machine Learning Techniques

Summary : This research compares different machine learning algorithms for detecting cyberattacks on a network. The algorithms evaluated include Support Vector Machines (SVM), Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), Random Forests (RF), and an unspecified "significant learning estimation."

The study finds that the significant learning method achieved the best overall results in identifying network intrusions. The authors plan to explore combining these algorithms with Apache Hadoop and SHIMMER advancements to analyze more attack types and improve accuracy.

Overall, the research highlights the potential of machine learning for network intrusion detection, allowing systems to learn from past attacks and improve their ability to identify new threats.[1]

Title : Predicting Network Attack Patterns in SDN using Machine Learning Approach

Summary : This research explores using machine learning to safeguard a specific type of network, a Software-Defined Network (SDN). The study's objective was to predict which devices within the network were most susceptible to attacks. By anticipating these vulnerable targets, the researchers aimed to prevent malicious actors from gaining access.

The approach involved training machine learning algorithms to analyze network data and pinpoint patterns that indicated a high risk of attack. The most successful algorithm, a Bayesian Network, achieved an impressive average accuracy of over 91%. This translates to accurately identifying potential targets in a vast dataset containing nearly 280,000 attacks.

The findings highlight the effectiveness of machine learning in predicting vulnerable devices within an SDN network. This allows for proactive security measures to be

implemented, safeguarding the network before attacks occur. Interestingly, the study also revealed that even low-probability threats shouldn't be ignored. As the researchers increased a specific threshold (alpha) used for flagging potential attacks, the prediction accuracy decreased. This suggests that security rules on the network controller should account for even these seemingly less likely threats.

In conclusion, this research demonstrates the potential of machine learning to significantly enhance network security within an SDN environment. By proactively identifying and protecting vulnerable systems, machine learning can play a crucial role in thwarting cyberattacks.[2]

Title : Machine Learning for Application-Layer Attack Detection

summary ; This research investigates using machine learning to combat a specific type of cyberattack: those targeting the application layer of a network. These attacks can slip past traditional defenses, making them especially dangerous.

The proposed solution involves training machine learning algorithms to analyze network traffic data collected from user activity. This data is then segmented using a graph-based approach, with plans to incorporate dynamic programming in the future. By analyzing the segmented data, the system can identify patterns that signal malicious activity at the application layer.

However, there are also challenges to consider. The accuracy of the system relies heavily on the quality and comprehensiveness of the training data. Additionally, there's a risk of the system mistakenly flagging legitimate traffic as malicious, which could disrupt normal network operations.

Beyond these considerations, the researchers acknowledge the need to evaluate the system's resource requirements, such as CPU, memory, and disk usage, to ensure it can

handle real-world network traffic volumes. Optimizing the machine learning algorithms' parameters is also crucial for maximizing detection accuracy.

Overall, this research presents a promising direction for application-layer attack detection using machine learning. By addressing the identified challenges and optimizing the system, it has the potential to become a valuable tool for fortifying network security.[3]

Title : Detection of cyber attack in network using machine learning technique

Summary : This research compared several machine learning algorithms for detecting cyberattacks on a network. The algorithms evaluated were Support Vector Machines (SVM), Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), Random Forests (RF), and a type of deep learning not specified in the text.

The study found that the deep learning method achieved the best results in identifying cyberattacks compared to the other algorithms. The authors plan to further explore using these algorithms along with Apache Hadoop and Spark technologies to analyze a wider range of attack types beyond port scans. They believe this approach can improve the accuracy of cyber intrusion detection.

Overall, the research highlights the potential of machine learning, particularly deep learning, for network security. By analyzing historical data containing information about past attacks, these algorithms can learn to identify patterns and predict future attacks with greater accuracy.[4]

Title : Cyber-attack detection in network traffic using machine learning

Summary : Our analysis identified network intrusion detection as a critical issue. We leveraged CRISP-DM, machine learning, and statistical software to explore a dataset and

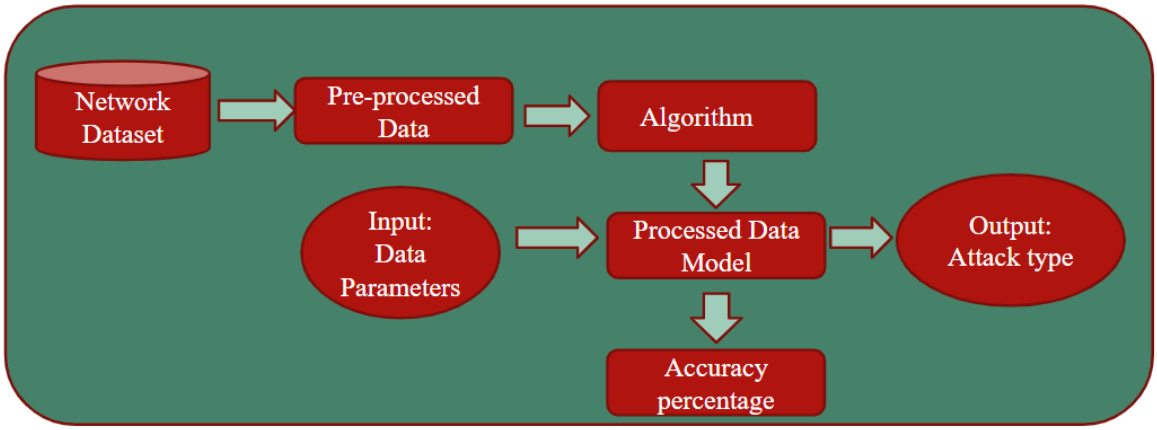
build intrusion detection models. The C5 decision tree emerged as the optimal model based on various performance metrics.[5]

Title : Detection of cyber attacks using machine learning

Summary : This research explores using machine learning to enhance network security. Different algorithms like Decision Trees (DT) were evaluated for their effectiveness in identifying malicious software. The study focused on choosing the right algorithm for the task and optimizing its performance. Results from various cybersecurity databases showed promising accuracy (up to 0.98) achieved by DT compared to other methods. This suggests that machine learning can be a powerful tool for building robust network security models.[6]

CHAPTER 3

PROPOSED METHODOLOGY



This diagram outlines a machine learning process for network intrusion detection. Raw network traffic data is collected, which might contain both regular activity and hidden cyberattacks. Before analysis, this data goes through preprocessing steps like cleaning and feature engineering to ensure it's suitable for the machine learning algorithm. The specific algorithm used isn't mentioned, but the options could be Support Vector Machines, Artificial Neural Networks, or even deep learning techniques. This algorithm is then trained on the preprocessed data, building a model that can classify new, unseen network traffic as normal or malicious. The model's performance is measured by accuracy, indicating how well it can differentiate between safe and harmful traffic. Finally, the ideal outcome is to identify the specific type of cyberattack present in the network, if any, allowing for targeted security measures. In essence, this machine learning approach analyzes network data, learns to recognize attack patterns, and ultimately bolsters network security by proactively detecting and preventing cyber threats.

CHAPTER 4

Tools & Techniques

Hardware/Software	**Developed by**	**Use of Hardware/Software in your project**
Software: Machine Learning Libraries (e.g., scikit-learn)	Scikit-learn developers	Libraries provide tools for data manipulation and feature engineering.
Machine Learning Libraries, Jupyter Notebook	Project Jupyter	Frameworks provide tools for building, training, and evaluating machine learning models. Jupyter Notebook can be used to define, train, and visualize the training process of the model.
Software: Visual Studio Code (IDE), Streamlit	Microsoft (Visual Studio Code)	Microsoft (Visual Studio Code) Visual Studio Code is a popular code editor that can be used for developing web applications with Streamlit. Streamlit is a Python library that allows you to create data apps for machine learning models.I use Visual Studio Code to write, test, and debug the code for deploying your model as a web application

CHAPTER 5

Implementation

4.1 Importing libraries

This fig 4.1 represents the importing of different libraries which required to execute.

```
In [70]: import numpy as np
         import pandas as pd
         import seaborn as sns
         import matplotlib.pyplot as plt
         import plotly.express as px
         from tensorflow.keras.layers import Dense, LSTM, Embedding, SimpleRNN
         from tensorflow.keras.models import Sequential
         from sklearn import metrics
         from sklearn.model_selection import cross_val_score
         from sklearn.preprocessing import MinMaxScaler
         from sklearn.preprocessing import LabelEncoder
         from sklearn.ensemble import RandomForestClassifier
         from sklearn.linear_model import LogisticRegression
         from sklearn.neighbors import KNeighborsClassifier
         from sklearn.tree import DecisionTreeClassifier
         from sklearn import linear_model

In [71]: from sklearn.model_selection import train_test_split
         from sklearn.metrics import mean_absolute_error,r2_score,mean_squared_error, accuracy_score
         from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
         from sklearn.preprocessing import RobustScaler
         from sklearn.svm import SVC
         import warnings
         warnings.filterwarnings('ignore')
```

Fig 4.1 Importing libraries

4.2 Analyse a dataset

This fig 4.2 displays the data from given dataset

```
In [111]: data = pd.read_parquet('UNSW_NB15_training-set.parquet')
          data.to_csv('UNSW_NB15_training-set.csv')

In [112]: df = pd.read_csv('UNSW_NB15_testing-set.csv')
          df

Out[112]:
```

	Unnamed: 0	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	...	trans_depth	response_body_len	ct_src_dport_ltm	ct_dst_s
0	0	0.121478	tcp	-	FIN	6	4	258	172	74.087490	...	0	0	1	
1	1	0.649902	tcp	-	FIN	14	38	734	42014	78.473370	...	0	0	1	
2	2	1.623129	tcp	-	FIN	8	16	364	13186	14.170161	...	0	0	1	
3	3	1.681642	tcp	ftp	FIN	12	12	628	770	13.677108	...	0	0	1	
4	4	0.449454	tcp	-	FIN	10	6	534	268	33.373825	...	0	0	2	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
175336	175336	0.000009	udp	dns	INT	2	0	114	0	111111.110000	...	0	0	24	
175337	175337	0.505762	tcp	-	FIN	10	8	620	354	33.612648	...	0	0	1	
175338	175338	0.000009	udp	dns	INT	2	0	114	0	111111.110000	...	0	0	3	
175339	175339	0.000009	udp	dns	INT	2	0	114	0	111111.110000	...	0	0	30	
175340	175340	0.000009	udp	dns	INT	2	0	114	0	111111.110000	...	0	0	30	

175341 rows × 37 columns

```
In [114]: attack_types = df['attack_cat'].unique()
```

Fig 4.2 Reading dataset

4.3 Unique attacks

This fig 4.3 it defines the unique attacks from dataset

```
In [114]: attack_types = df['attack_cat'].unique()
          attack_types

Out[114]: array(['Normal', 'Backdoor', 'Analysis', 'Fuzzers', 'Shellcode',
                 'Reconnaissance', 'Exploits', 'DoS', 'Worms', 'Generic'],
                dtype=object)
```

Fig 4.3 Dataset unique attacks

4.4 Summary of a dataframe

This fig 4.4 it shows information and data type of dataset

```
In [119]: import pandas as pd
          df_train.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175341 entries, 0 to 175340
Data columns (total 36 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   Unnamed: 0         175341 non-null  int64
 1   dur                175341 non-null  float64
 2   proto              175341 non-null  object
 3   service            175341 non-null  object
 4   state              175341 non-null  object
 5   spkts              175341 non-null  int64
 6   dpkts              175341 non-null  int64
 7   sbytes             175341 non-null  int64
 8   dbytes             175341 non-null  int64
 9   rate               175341 non-null  float64
 10  sload              175341 non-null  float64
 11  dload              175341 non-null  float64
 12  sloss              175341 non-null  int64
 13  dloss              175341 non-null  int64
 14  sinpkt             175341 non-null  float64
 15  dinpkt             175341 non-null  float64
 16  sjit               175341 non-null  float64
 17  djit               175341 non-null  float64
 18  swin               175341 non-null  int64
 19  stcpb              175341 non-null  int64
 20  dtcpb              175341 non-null  int64
 21  dwin               175341 non-null  int64
 22  tcprtt             175341 non-null  float64
 23  synack             175341 non-null  float64
 24  ackdat             175341 non-null  float64
 25  smean              175341 non-null  int64
```

Fig 4.4 Dataset info

4.5 Data Formatting

This fig 4.5 it changing the data one format to another format;

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder


# Initialize the LabelEncoder
le = LabelEncoder()

# Apply LabelEncoder to the specified columns
df_train['proto'] = le.fit_transform(df_train['proto'])
df_train['service'] = le.fit_transform(df_train['service'])
df_train['state'] = le.fit_transform(df_train['state'])
df_train['attack_cat'] = le.fit_transform(df_train['attack_cat'])
```

```
# print(sorted(attack_types))
# print(sorted(df_train['attack_cat'].unique()))

num=0
for i in sorted(attack_types):
    print(i,":",num)
    num+=1
```

```
Analysis : 0
Backdoor : 1
DoS : 2
Exploits : 3
Fuzzers : 4
Generic : 5
Normal : 6
```

Fig 4.5 Encoding Categorical values of train dataset

4.6 Data Formatting Train Dataset

This fig 4.6 it changing dataset categorical values of test dataset

```
In [139]: le = LabelEncoder()
          df_test['proto']=le.fit_transform(df_test['proto'])
          df_test['service']=le.fit_transform(df_test['service'])
          df_test['state']=le.fit_transform(df_test['state'])
          df_test['attack_cat']=le.fit_transform(df_test['attack_cat'])
```

```
In [133]: print(df_test.shape)
          print(df_test.isnull().values.any())
```

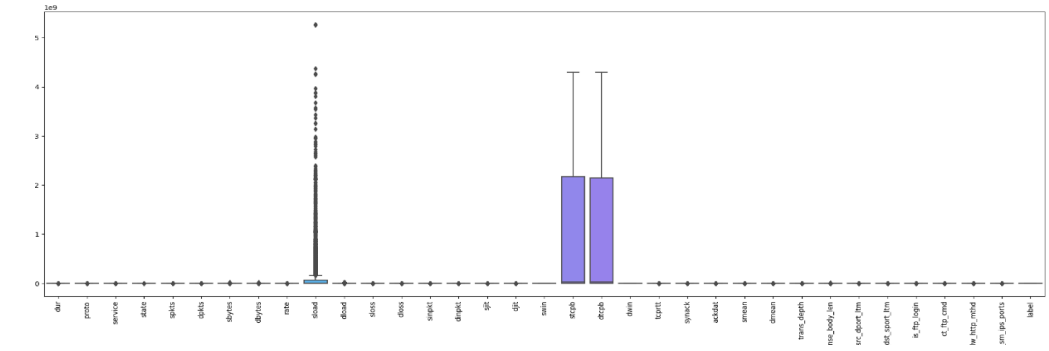
```
(175341, 36)
False
```

Fig 4.6 Encoding Categorical values of test dataset

4.7 Outliers

This Fig 4.7 An outlier is an observation that lies an abnormal distance from other values in a random sample from a population

```
]: plt.figure(figsize = (25,8))
   u = sns.boxplot(palette = 'cool', data=data_train)
   u.set_xticklabels(u.get_xticklabels(),rotation=90);
```




```
In [148]: fig = px.box(data_test, y='sload')
          fig.show()
```

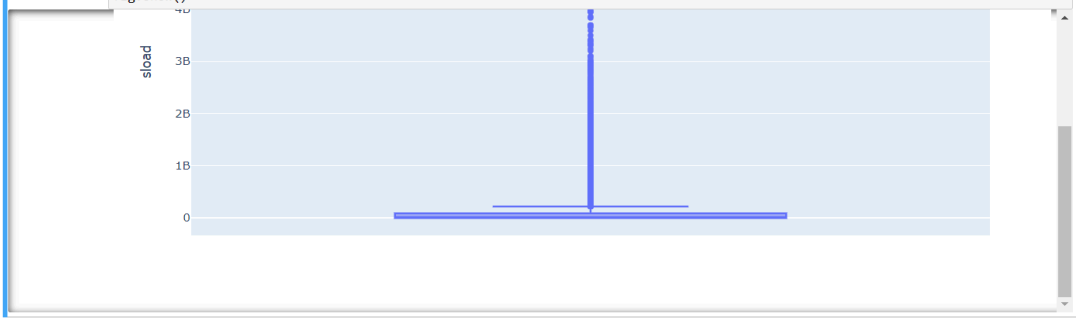


Fig 4.7 outliers

4.8 Removing outlier

This fig4.8 Removing outlier Outliers can be treated in different ways, such as trimming, capping, discretization, or by treating them as missing values

```
In [157]: data_train1 = df_train.copy()
          data_train_wthout1 = data_train1.drop(df_train[df_train['sload']>162400000].index)
          data_train_wthout = data_train_wthout1.drop(['attack_cat'], axis=1)

In [150]: data_train_wthout.shape

Out[150]: (75617, 34)
```

Fig4.8 Removing outlier

4.9 Train Test split

This Fig4.9 A train test split is when you split your data into a training set and a testing set. The training set is used for training the model, and the testing set is used to test your model.

```
In [134]:
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import RobustScaler
from sklearn.model_selection import train_test_split
x = df_train.drop(['attack_cat'], axis=1).values
scaler = MinMaxScaler()
x = scaler.fit_transform(x)
y = df_train['attack_cat'].values

x_train , x_test , y_train , y_test = train_test_split(x,y , test_size= 0.2 , random_state=42)

ro_scaler = RobustScaler()
x_train = ro_scaler.fit_transform(x_train)
x_test = ro_scaler.fit_transform(x_test)

In [140]:
xl_test = df_test.drop(['attack_cat'], axis=1).values
yl_test = df_test['attack_cat'].values

In [144]:
xl_test

Out[144]:
array([[0.000000e+00, 1.214780e-01, 1.130000e+02, ..., 0.000000e+00,
        0.000000e+00, 0.000000e+00],
       [1.000000e+00, 6.499020e-01, 1.130000e+02, ..., 0.000000e+00,
        0.000000e+00, 0.000000e+00],
       [2.000000e+00, 1.623129e+00, 1.130000e+02, ..., 0.000000e+00,
        0.000000e+00, 0.000000e+00],
       ...,
       [1.753380e+05, 9.000000e-06, 1.190000e+02, ..., 0.000000e+00,
        0.000000e+00, 0.000000e+00],
       [1.753390e+05, 9.000000e-06, 1.190000e+02, ..., 0.000000e+00,
```

Fig 4.9 Train Test split

CHAPTER 6

RESULT AND DISCUSSION

5.1 Logistic Regression

This fig 5.1 Logistic regression is a supervised machine learning algorithm that accomplishes binary classification tasks by predicting the probability of an outcome, event, or observation

```
In [168]: from sklearn.linear_model import LogisticRegression
          from sklearn.preprocessing import StandardScaler
          from sklearn.pipeline import make_pipeline
          from sklearn.model_selection import cross_val_score
          import numpy as np
          logr = LogisticRegression()
          logr_cross = fit_and_evaluate(logr, xx_train , xx_test , yy_train , yy_test)

          print('Logistic Regression Performance on the test set: Cross Validation Score = %0.4f' % logr_cross)

          Logistic Regression Performance on the test set: Cross Validation Score = 0.7652

In [169]: from sklearn.metrics import accuracy_score
          y_pred = logr.predict(xl_test)
          print("Accuracy on test dataset: ", accuracy_score(yl_test, y_pred))

          Accuracy on test dataset:  0.5947794812851078
```

Fig 5.1 Logistic Regression

5.2 Confusion Matrix

This fig 5.2 The confusion Matrix gives a comparison between actual and predicted values

```
In [170]: cm = confusion_matrix(yl_test, y_pred, labels=logr.classes_)
          disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=logr.classes_)
          disp.plot()
          plt.show()
```

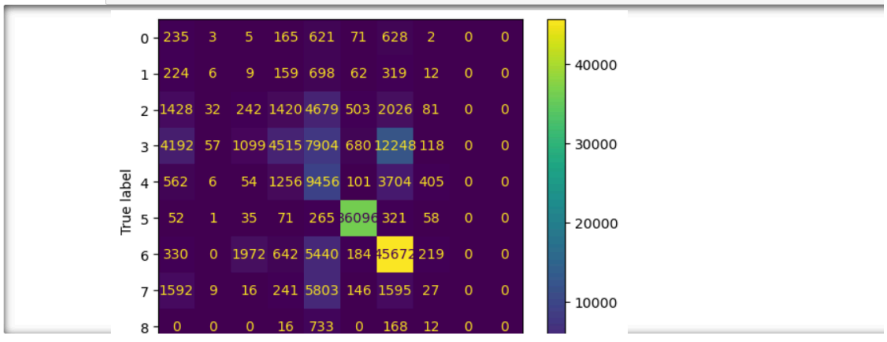


Fig 5.2 Confusion Matrix

5.3 k-nearest neighbors

This fig 5.3 KNN is non-parametric, supervised learning classifier, which uses proximity to make classifications or predictions about the grouping of an individual data point.

KNN:

```
In [171]: from sklearn.neighbors import KNeighborsClassifier
          knn = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)
          knn_cross = fit_and_evaluate(knn, xx_train , xx_test , yy_train , yy_test)

          print('KNN Performance on the validation set: Cross Validation Score = %0.4f' % knn_cross)

          KNN Performance on the validation set: Cross Validation Score = 0.7895

In [172]: yl_pred = knn.predict(xl_test)
          print("Accuracy on test dataset: ", accuracy_score(yl_test, yl_pred))

          Accuracy on test dataset:  0.4207745499712649
```

Fig 5.3 k-nearest neighbors (KNN)

5.4 Confusion Matrix

This fig 5.4 The confusion Matrix gives a comparison between actual and predicted values

```
In [173]: cm = confusion_matrix(yl_test, y_pred, labels=knn.classes_)
          disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=knn.classes_)
          disp.plot()
          plt.show()
```

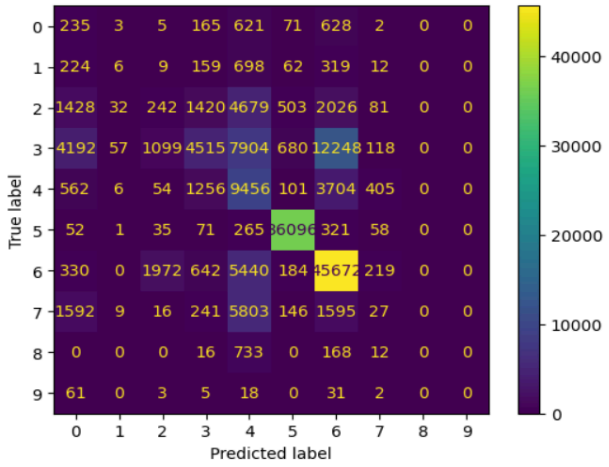


Fig 5.4 Confusion Matrix

5.5 DecisionTree

This fig 5.5 A decision tree is a non-parametric supervised learning algorithm, which is utilized for both classification and regression tasks.

```
# Predict on the training data
y_train_pred = dt.predict(xx_train)

# Evaluate accuracy on the training set
try:
    train_accuracy = accuracy_score(yy_train, y_train_pred)
    print("Accuracy on train dataset: ", train_accuracy*100)
except Exception as e:
    print("Error occurred while computing training accuracy:", e)
```

```
Accuracy on train dataset:  100.0
```

In [176]:

```
from sklearn.metrics import accuracy_score

# Evaluate accuracy on the test set
try:
    accuracy = accuracy_score(yy_test, y_pred)
    print("Accuracy on test dataset: ", accuracy)
except Exception as e:
    print("Error occurred while computing accuracy:", e)
```

```
Accuracy on test dataset:  0.8610892862754957
```

Fig 5.5 DecisionTree

5.6 Confusion Matrix

This fig 5.6 The confusion Matrix gives a comparison between actual and predicted values

In [179]:

```
# Compute confusion matrix
cm = confusion_matrix(yy_test, y_pred, labels=dt.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=dt.classes_)

# Display the confusion matrix
disp.plot()
plt.show()
```

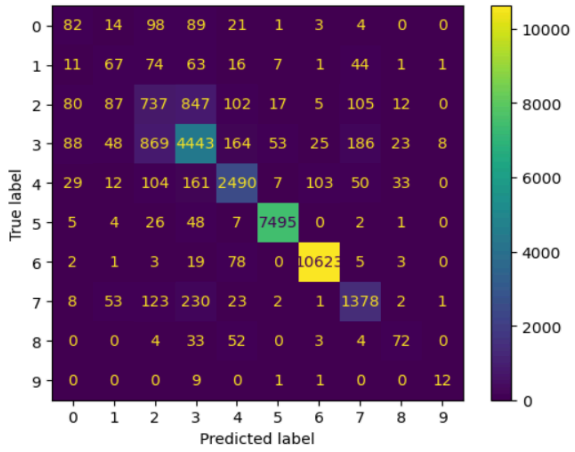


Fig 5.6 Confusion Matrix

5.7 Streamlit

This fig 5.7 Streamlit is a promising open-source Python library, which enables developers to build attractive user interfaces in no time.

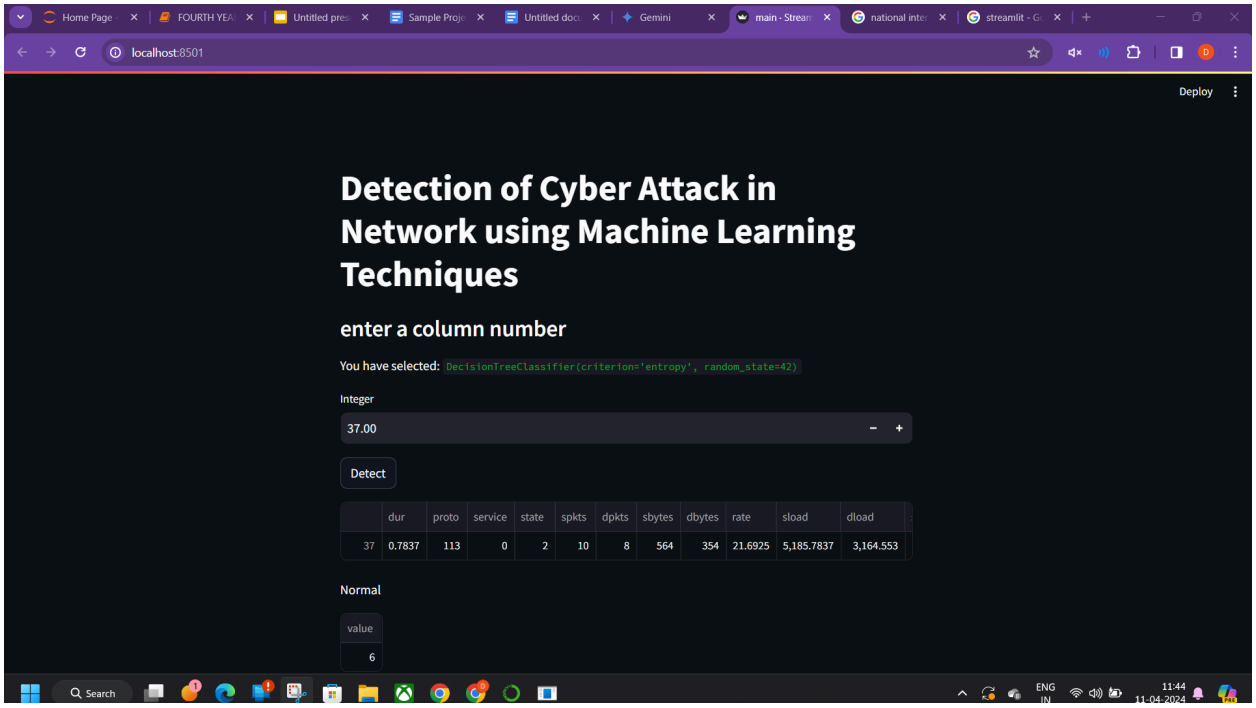


Fig 5.6 5.7 Streamlit

CHAPTER 7

REFERENCE

6.1 Journal Reference

[1]Uyyala P. DETECTION OF CYBER ATTACK IN NETWORK USING MACHINE LEARNING TECHNIQUES. Journal of interdisciplinary cycle research. 2022;14(3)

[2]Reddy D, Sajjan BT, Anusha M, Sadiq SJ, Shambulingappa HS. Detection of Cyber Attack in a Network using Machine Learning Techniques. International Journal of Advanced Scientific Innovation. 2021 May 27

[3] Nanda S, Zafari F, DeCusatis C, Wedaa E, Yang B. Predicting network attack patterns in SDN using machine learning approach. In2016 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN) 2016 Nov 7

[4] M. Baykara, R. Das¸, and I. Karadogan, "Bilgi guvenligi sistemlerinde kullanilan araclarin incelenmesi," in 1st International Symposium on Digital Forensics and Security (ISDFS13), 20139.

[5] Rashmi T V. "Predicting the System Failures Using Machine Learning Algorithms". International Journal of Advanced Scientific Innovation, vol. 1, no. 1, Dec. 2020.

[6] S. Robertson, E. V. Siegel, M. Miller, and S. J. Stolfo, "Surveillance detection in high bandwidth environments," in DARPA Information Survivability Conference and Exposition, 2003. Proceedings, vol. 1. IEEE, 2003.

[7]Shaukat K, Luo S, Chen S, Liu D. Cyber threat detection using machine learning techniques: A performance evaluation perspective. In2020 international conference on cyber warfare and security (ICCWS) 2020 Oct 20

[8]Almulla K. Cyber-attack detection in network traffic using machine learning.(2022)

6.2 Web Reference

[1]https://www.youtube.com/watch?v=51sAIVs65nM&ab_channel=MallikarjunaInfosys

[2]https://www.youtube.com/watch?v=hWbntQyMrI0&ab_channel=PankajVarmaDendukuri

[3]https://www.kaggle.com/code/shahtiham/unsw-nb15/notebook#Logistic-Regressoin-after-removing-outlier:

CHAPTER 7

WORKLOG

Internship Start Date : February 12, 2024

DAY	DATE	TASK DONE
Day 1 to Day 2	11/02/2024 to 12/02/2024	Use Google Scholar, ScienceDirect, IEEE Xplore (check for free trials/institutional access) or arXiv to search for "network intrusion detection" AND "machine learning" papers.
Day 3 to Day 13	13/02/2024 to 23/02/2021	Actively engage with research papers (skim, deep read, take notes, visualize) focusing on core concepts and considering paper age.
Day 14 to Day 19	24/02/2024 to 29/02/2024	Dive into data preprocessing for machine learning via online courses, tutorials (Scikit-learn etc.) and books (Brownlee, Géron) focusing on network intrusion detection specifics.
Day 20 to Day 23	01/03/2024 to 04/03/2024	Clean Network intrusion detection with machine learning involves cleaning, normalizing, and crafting features from network traffic data using tools like scikit-learn.
Day 24 to Day 26	05/03/2024 to 07/03/2024	Apply LabelEncoder to specified columns (e.g., "column1", "column2") for categorical encoding using apply or dictionary comprehension.
Day 27 to Day 30	08/03/2024 to 12/03/2024	Leverage cross_val_score to assess your machine learning model's effectiveness across multiple data splits
Day 31	13/03/2024	Employ scikit-learn's LogisticRegression for binary classification tasks in your

		network intrusion detection project
Day 32	14/03/2024	A confusion matrix with more FP and FN indicates frequent incorrect identifications and missed true instances.
Day 33 to Day 33	15/03/2024 to 16/03/2024	Address outliers consistently across train and test data using techniques like IQR (Interquartile Range) for anomaly detection and removal.
Day 35 to Day 42	17/03/2024 to 24/03/2024	Build and train models (e.g :Logistic Regression, Dc and Knn) on preprocessed network traffic data to identify and classify cyberattacks.
Day 43	25/03/2024	Scrutinize your code to identify and rectify errors, improve efficiency, and guarantee the robustness of your network intrusion detection system.
Day 44	26/03/2024	Utilize cross_val_score (e.g., from scikit-learn) to rigorously evaluate your network intrusion detection model's performance across various data splits, fostering generalizability and mitigating overfitting.
Day 46	28/03/2024	After making code modifications or data adjustments, re-execute your machine learning models
Day 47 to Day 49	29/03/2024 to 31/03/2024	Leverage trained models (e.g., Logistic Regression, Random Forest) on unseen network traffic data to forecast potential cyberattack types, bolstering network security.
Day 50 to Day 56	01/03/2024 to 07/03/2024	Build interactive visualization app on Streamlit
Day 57	08/03/2024	Compile a report detailing your methodology (data, preprocessing, models), results (performance metrics, attack type predictions), and potential future improvements for a comprehensive record of your network security project.